Smart Room Monitor: An RTOS-Based Environmental Sensing and Alert System

Rifa Tasfia | rifatasf
Arnav Nanda | arnavnan

1. Problem title and statement

The problem: Comfort, sleep, health, and productivity are affected by indoor environmental conditions, but people generally don't have real-time insight into them. In a dorm room, for example, things such as heat, high humidity, excessive darkness, or poor air quality, which may make you drowsy or unable to concentrate, might go unnoticed until it's too late. Long-term excessive humidity can even cause mold, and stale air quickly accumulates in confined spaces. Traditional thermostats measure temperature only at one point and do nothing else. Advanced smart monitors that measure several conditions and react to them cost a lot of money and rely on cloud service.

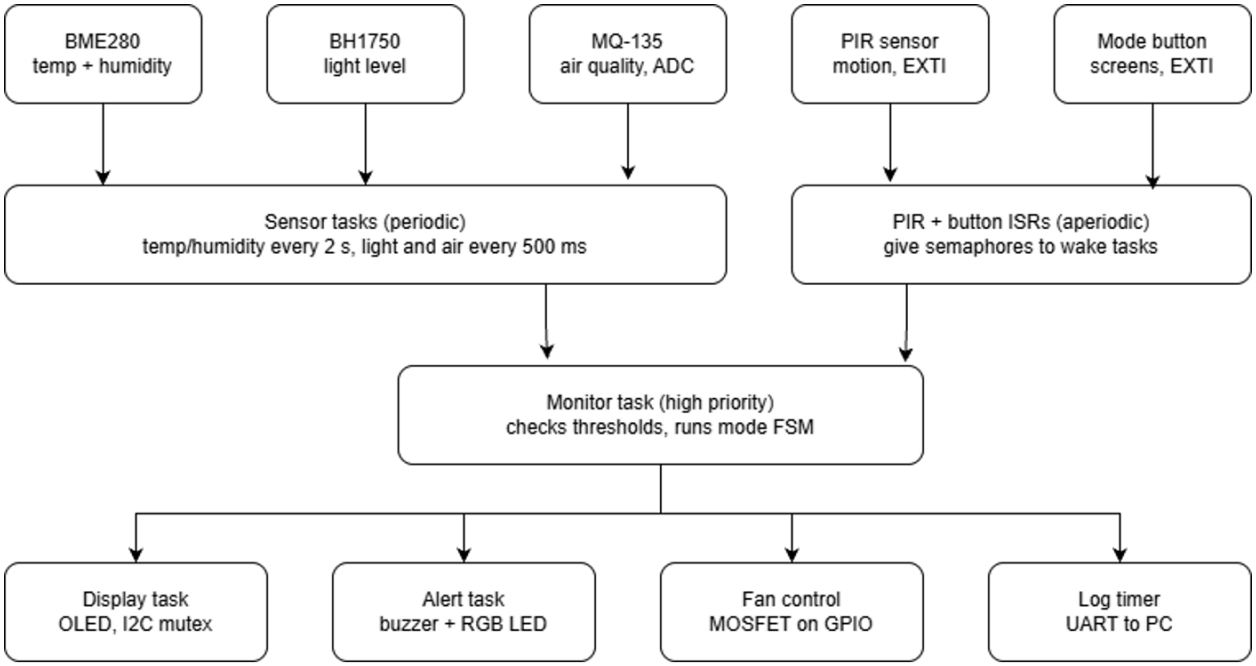
How the product solves it: Smart Room Monitor is an affordable, self-contained product that monitors several room parameters simultaneously and reacts automatically to any discomfort. Unlike other products, it not only informs the user about these conditions but reacts, warning him or her and automatically ventilating the room, if needed. It also detects whether anybody is in the room or not, thus saving power.

What the system is supposed to do:

- Monitor temperature, humidity, light, and air quality in real time.
- Display the live data on an OLED screen; also provide a button to toggle through the display and turn off the alarm sounds.
- Categorize the room’s environmental status into three states: Normal, Warning, and Alert; display the current state using an RGB LED, colored green, yellow, and red accordingly.
- Generate an alarm sound when any of the readings exceeds its threshold value.
- Automatically turn ON the 12V fan in case of elevated temperature or air quality readings.
- Detect occupancy in the room via a PIR sensor; automatically switch to Sleep mode after some pre-defined idle period.
- Log the readings to a computer through UART communication.

Since monitoring, decision making, display, and alarms occur simultaneously and at various rates, the design is based on FreeRTOS running on an STM32 Nucleo microcontroller.

2. Simple block diagram of the system with description



Hardware Description and Interaction:

1. **STM32 Nucleo (main controller)**: Runs FreeRTOS and application software. It reads data from all sensors, determines the state of the room and controls all outputs.
2. **I2C Bus**: There is only one I2C bus that BME280 (temperature and humidity), BH1750 (light) and SSD1306 OLED use. Since there are multiple devices connected to the bus, access is protected by the mutex in RTOS.
3. **MQ-135 (air quality sensor)**: Produces analog voltage measured by ADC of STM32. The output voltage may be greater than 3.3V; thus, it goes through the voltage divider.
4. **PIR Sensor and Push Button**: Both PIR sensor and push-button are connected to GPIOs configured as external interrupts (EXTI). Motion causes waking the system from Sleep mode while push-button changes the screen on LCD or mutes the alarm.
5. **Piezo Buzzer**: Controlled in PWM mode by hardware timer that produces alert sounds.
6. **RGB LED**: Three GPIOs with current limiting resistors indicate the state of the system.
7. **MOSFET module and Fan**: One GPIO activates the MOSFET module and controls 12V fan. Microcontroller doesn't carry any current of the fan.
8. **PC Connection**: Data is logged over UART via USB virtual COM port (ST-LINK).
9. **Power**: A wall adapter that is 12V and 2A powers the fan and provides power to the buck converter, which reduces the voltage to 5V for the Nucleo board and the MQ-135 heater. The Nucleo board's on-board voltage regulator provides 3.3V for the rest of the components.

3. COMPONENTS LIST

Component	Usage	Status
NUCLEO-F072RB	Microcontroller	On Hand
BME280	Temperature/Humidity Sensor	Ordered
BH1750	Light Intensity Sensor	On Hand
MQ-135	Air Quality Sensor	Ordered
Buck Converter	Provides Proper Voltage for MQ-135	On Hand
PIR Motion Sensor	Motion Detector	On Hand
Push Buttons	Switch Through Pages on OLED	On Hand
GME12864	OLED Displays Status and Readings	On Hand
Piezo Buzzer	Provides Audible Alerts	On Hand
RGB LED	Indicates Room Status	On Hand
MOSFET	Allows the microcontroller to control the fan	On Hand
12V DC Fan	Allows for Ventilation	On Hand
12V, 2A DC Wall Adapter	Power Supply for hardware	On Hand
Protoboard	Solder Component Connections	On Hand
Resistors	Limit Current/ Adjust Voltage	On Hand
Jumper Cables	Make Connections	On Hand

DEVELOPMENT ENV/IDE	STM32CubeIDE	INSTALLED
DEBUGGER	ST-LINK DEBUGGER	INSTALLED

4. Description of RTOS components that will be implemented

a. Identify RTOS being used

For our system, we plan on using FreeRTOS on the NUCLEO-F072RB. We will add FreeRTOS to our STM32CubeIDE project and include the required header files in our code. FreeRTOS will help organize and schedule the different tasks for our program.

b. RTOS tasks/components that will be implemented

Sensor Tasks: These tasks will collect temperature, humidity, light, and air quality readings from the sensors at timed intervals.

Monitoring Task: This task will check the sensor readings to confirm if the room conditions are within the set limits.

Alert Task: This task will activate the RGB LED and buzzer when the sensor readings reach certain limits.

Fan Control: The fan will automatically turn on when the temperature becomes too high, or air quality becomes too poor.

Display Task: This task will show the sensor readings on the OLED display. The user will also be able to switch between screens using a push button.

Motion Detection: The PIR sensor will detect movement in the room. If no movement is detected for a certain amount of time, the system will enter Sleep mode.

Other FreeRTOS Components: We plan on using queues to share sensor readings between tasks, a mutex to manage the I2C bus, and semaphores for button presses and motion detection. Task priorities will help manage which task runs first.

c. Explain why an RTOS is appropriate for the system

FreeRTOS is appropriate for our Smart Room Monitor because the system needs to handle multiple tasks, such as collecting sensor readings, updating the display, and activating alerts. FreeRTOS lets us schedule these tasks and prioritize important tasks. Therefore, the system can respond to changes in room conditions without delays.